CVPR 2027 SUBMISSION · SUPPLEMENTARY PROJECT PAGE

SEPM: Self-Evolving Procedural Memory Grounded in Task Graphs for Multi-Agent Collaboration

Anonymous Authors
Under double-blind review

Main paper | Code in supplementary | 7 benchmarks | AutoGen · DyLAN · CoELA

*Team memory that records not only **what** to do, but also **what must already be true** before each step.*

6 / 6
main settings ranked first against 7 memory baselines

+22.86
three-benchmark mean under AutoGen (37.74 → 60.60)

47.61 → 59.53
ImplicitMemBench overall 82% of the oracle's gain

−8.13 to −16.91
points when task graphs are replaced by textual prerequisites

ABSTRACT In a team, whether an agent can take its next step hinges on what its teammates have completed. Existing memories store procedures without these **cross-agent prerequisites**, so experience can neither be checked during execution nor separated from incidental co-occurrence. **SEPM** uses **task dependency graphs** as one representation for both coordination and memory: executable steps are nodes and prerequisites are edges. During execution, a step runs only after its prerequisites are verified, and goal, plan, and state divergence is corrected on a shared blackboard; afterwards, verified steps and edges are distilled into versioned procedures under evidence gates.

1 Why team memory needs a task graph

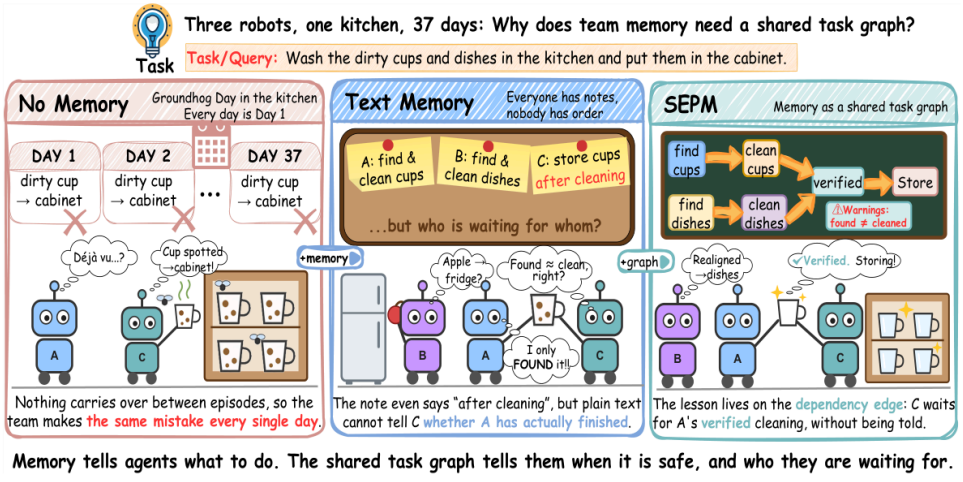


Figure 1. Three robots wash the dirty cups and dishes and store them in a cabinet. Without memory, the team stores a dirty cup in every episode. Textual memory records "after cleaning", but no agent can check it. SEPM stores the prerequisite as a dependency edge, so C stores a cup only after A's cleaning is verified.

Q1 · How should experience act on execution?

Text changes behavior only if each agent interprets it correctly, and no single agent can confirm a prerequisite that spans several agents.

SEPM: store prerequisites as graph edges that the executor **checks** before dispatching a step.

Q2 · What should a team retain?

The order of interleaved steps does not reveal which steps enabled later ones, so memory evolution is a credit-assignment problem.

SEPM: keep only the steps and edges that **verification records support**, not temporal precedence.

2 Method: one graph for coordination and memory

A retrieved procedure is instantiated as a shared task graph. The same graph constrains which steps may run, anchors the alignment of every agent, and decides what the team retains after the task.

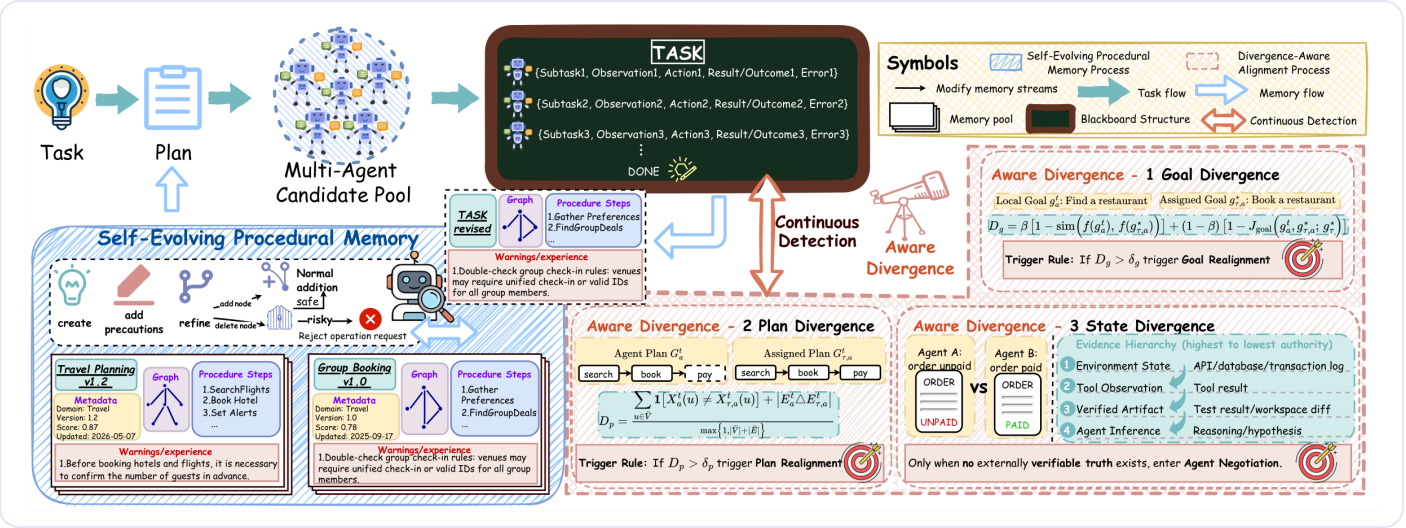


Figure 2. Overview of SEPM. Retrieved procedures become task graphs on a shared blackboard, which supports goal, plan, and state alignment during execution and drives procedure evolution afterwards.

① Shared blackboard

Retrieves up to $K = 5$ applicable procedures and plans one task graph from them. A step is dispatched only when all its prerequisites are verified by **external evidence** (environment state, tool observation, or verified artifact), never by agent inference alone.

`models.py · storage.py · retrieval.py`

② Divergence-aware alignment

Compares each agent's local view with its reference subgraph. **Goal:** text dissimilarity + LLM judge. **Plan:** action mismatches and edges present in only one plan. **State:** conflicting evidence, resolved by an authority hierarchy. Corrections propagate to dependent steps.

`divergence.py · alignment.py · evidence.py`

③ Graph-grounded evolution

Keeps only verified steps and edges whose source established a condition **checked before** the target ran. Revisions are committed through evidence gates as new versions; lessons such as "found ≠ cleaned" become conditional warnings.

`episode.py · service.py · safety.py`

Prerequisite-constrained execution

$$\text{ready}(w) = \bigwedge_{(u,w)\in E} \mathbb{1}[\,\Sigma \vDash u\,]$$

A step becomes ready only after the outcome of every prerequisite is confirmed by the verified state Σ.

Evidence-gated evolution

$$\text{Accept}(S') = G_{\text{safety}} \wedge G_{\text{state}} \wedge G_{\text{dep}} \wedge G_{\text{qual}} \wedge [\,\bar{u}(S') \geq 0\,]$$

No unsafe action, consistent with the verified state, ≥ 65% evidence-supported edges, a qualified trial, and non-negative net benefit.

One task through SEPM

Retrieve & plan
Compatible procedures are retrieved and bound into one task graph on the blackboard.

Execute
Agents run ready steps and append (step, observation, action, result, error) records.

Realign
Goal, plan, or state divergence realigns the affected agents and their descendants.

Evolve
Supported steps and edges form new procedures or gated revisions; warnings are attached.

Commits are atomic (compare-and-swap on the head version), and no operation deletes a procedure or a version, so every revision can be audited and rolled back.

3 Case study: cleaning and storage

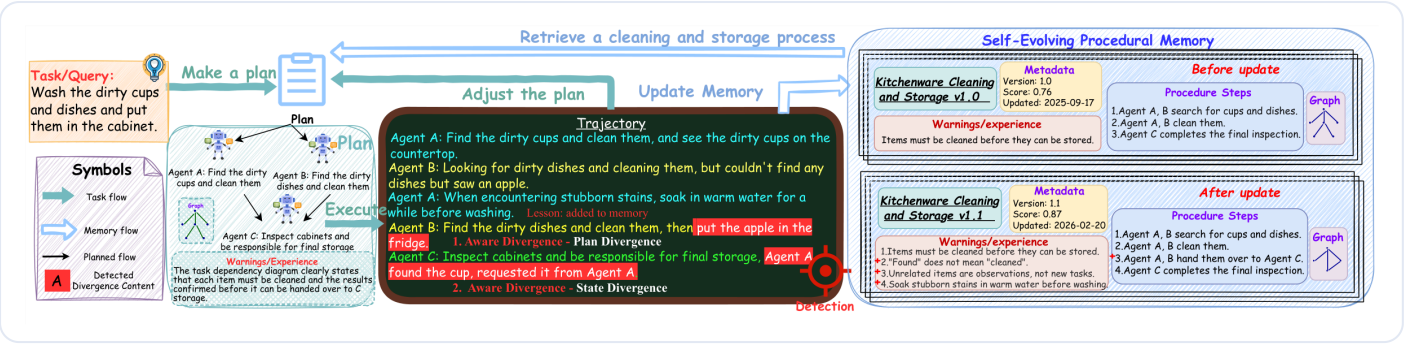


Figure 3. One task from retrieval to the revised procedure. Left: planning and execution, with the trajectory on the blackboard and the detected divergences. Right: the stored procedure before and after the update. Plan divergence (B stores an apple) and state divergence (C requests a cup that is found but not cleaned) trigger realignment, and the procedure is revised from v1.0 to v1.1.

4 Main results

7 BASELINES · 2 HOSTS · 3 BENCHMARKS

	AutoGen				DyLAN			
Memory	**ALFWorld**	**WebArena**	**OfficeB.**	**Avg.**	**ALFWorld**	**WebArena**	**OfficeB.**	**Avg.**
No added memory	59.45	12.44	41.33	37.74	53.98	14.90	33.56	34.15
Generative Agents	85.32	15.27	38.67	46.42	65.17	15.27	32.89	37.78
MetaGPT	77.86	25.99	45.11	49.65	49.50	22.41	35.22	35.71
G-Memory	83.08	30.30	49.67	54.35	62.19	25.49	42.44	43.37
LEGOMem	84.08	31.65	51.33	55.69	63.68	26.35	43.78	44.60
MATM	81.34	29.31	48.44	53.03	61.44	24.75	41.22	42.47
AWM†	80.10	26.48	46.44	51.01	60.70	23.40	38.89	41.00
Memp†	82.09	27.34	47.22	52.22	62.44	24.14	39.56	42.05
SEPM (ours)	**89.05**	**38.30**	**54.44**	**60.60**	**68.41**	**29.80**	**47.11**	**48.44**
vs. strongest baseline	**+3.73**	**+6.65**	**+3.11**	**+4.91**	**+3.23**	**+3.45**	**+3.33**	**+3.84**

Table 1. Task performance (%) under two hosts. Every memory starts empty and is built online over the same task sequence with the same retrieval count and context budget; all modules use `gpt-5-mini`. ALFWorld and OfficeBench: mean of three shuffled task orders; WebArena: one run. SEPM is best in every column; second best underlined. † Single-agent memory adapted to the team with role-specific allocation.

Significant in every setting

All six gains over no added memory are significant ($p < 10^{-4}$), and the margin over the strongest baseline in four ($p < 0.05$). The strongest baseline changes with the environment, yet SEPM leads in all of them.

Coordination or memory?

Inside SEPM's own blackboard and alignment, baseline memories add only **0.51–2.30** points, whereas SEPM's memory adds **4.86**: what the team stores matters beyond how it coordinates.

	TDW-MAT (TR ↑)			C-WAH
CoELA host	**Food**	**Stuff**	**Avg.**	**Steps ↓**
No added memory	68.33	58.33	63.33	92.4
G-Memory	72.50	61.67	67.08	87.1
AWM†	70.83	60.00	65.42	88.5
LEGOMem	73.33	62.50	67.92	85.6
SEPM (ours)	**79.17**	**68.33**	**73.75**	**79.8**

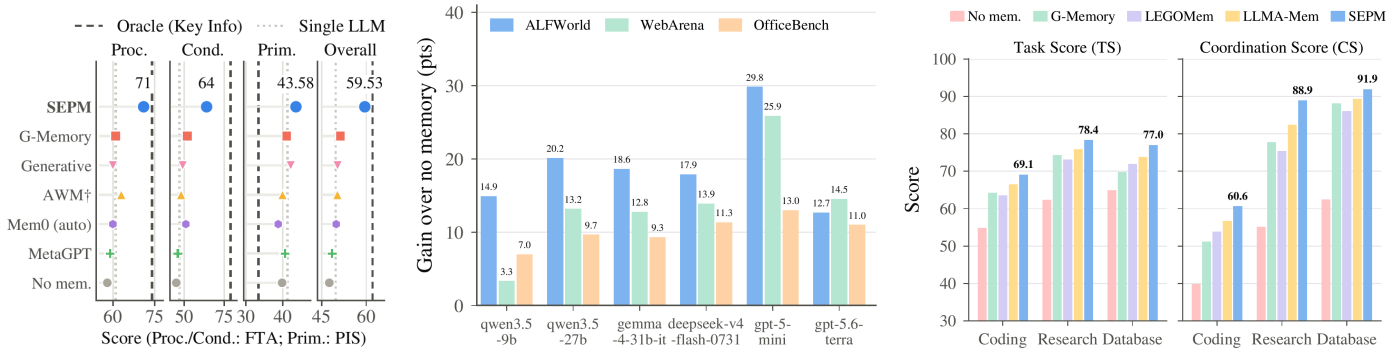
Table 2. Embodied cooperation with CoELA. TR: transport rate (%, three seeds); Steps: average steps per task (five seeds).

Physical handoffs

In TDW-MAT and C-WAH, agents act from visual observations and prerequisites are objects passed between agents. SEPM raises the transport rate by **10.42** points and saves **12.6** steps per task over the host without memory, ahead of LEGOMem by 5.83 points and 5.8 steps.

5 Across benchmarks, models, and embodiment

Beyond the main table, we test whether stored experience actually changes later behavior (ImplicitMemBench), whether the gain survives a change of execution model, and whether coordination itself improves (MultiAgentBench).



(a) ImplicitMemBench (b) Gain across execution models (c) MultiAgentBench

Figure 4. SEPM across benchmarks and models, under AutoGen. (a) ImplicitMemBench; dashed and dotted lines mark the oracle and a single LLM call. (b) Gain over no added memory with different execution and memory models. (c) Task Score (TS) and Coordination Score (CS) on MultiAgentBench.

IMPLICIT MEMORY

47.61 → 59.53

Highest non-oracle score with `gpt-5-mini` (82% of the oracle's gain); no baseline gains more than 3.75. With Qwen2.5-7B: 41.12 → 50.98.

EXECUTION MODELS

18 / 18 pairs positive

Gain of 3.33–29.85 points; even `gpt-5.6-terra`, the strongest without memory, gains 12.69 / 14.54 / 11.00.

COORDINATION QUALITY

+2.57–6.45 CS

Over the strongest alternative, LLMA-Mem, on MultiAgentBench coding, research, and database, with +2.44–3.17 TS.

6 Memory evolution and transfer

Because only evidence-supported structure is retained, the pool keeps improving along the task sequence and carries over to unseen task types, other hosts, and other models without any update.

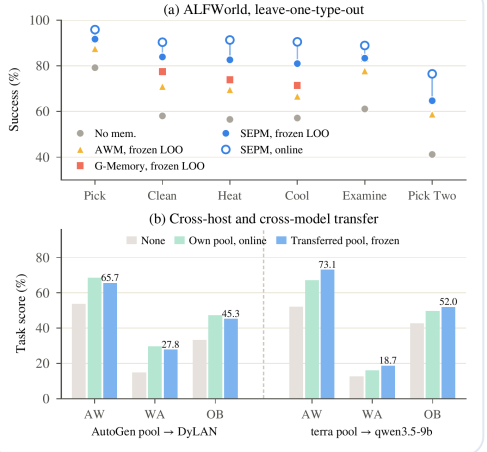


Figure 5. Transfer of frozen SEPM pools (one run). (a) ALFWorld leave-one-type-out; hollow markers show online SEPM. (b) Transfer across hosts and models.

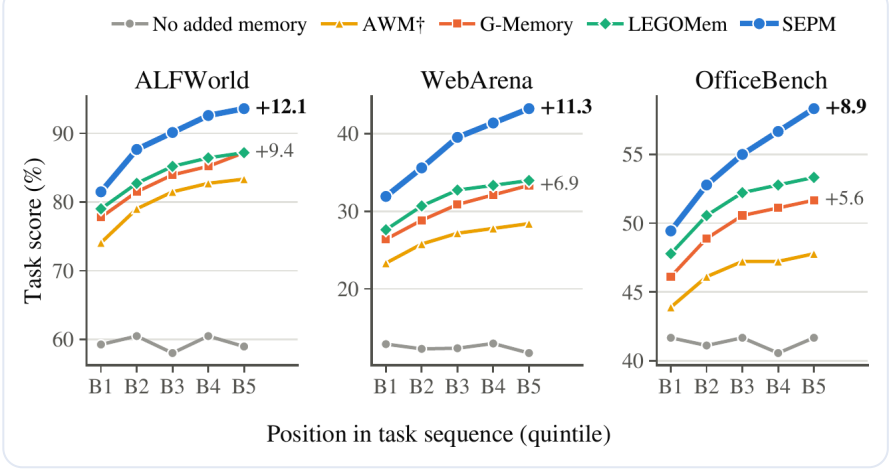


Figure 6. Task score along the task sequence under AutoGen, in five equal segments. Without memory, scores stay flat; SEPM gains **12.11 / 11.31 / 8.89** points from B1 to B5, versus 9.40 / 6.95 / 5.56 for G-Memory.

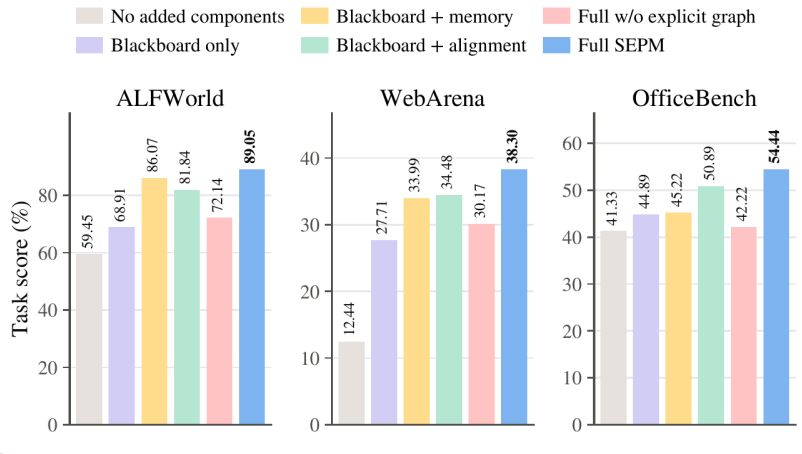
Transfer to unseen task types and hosts

A pool frozen before the held-out ALFWorld type reaches **82.09%** (G-Memory 77.61%, AWM 72.39%; online SEPM 89.55%). An AutoGen pool recovers **80.0 / 86.8 / 85.7%** of DyLAN's online gain without any update.

From strong to weak models

A pool built with `gpt-5.6-terra` lifts `qwen3.5-9b` by **5.97 / 2.71 / 2.33** points above its own online pool: the retained structure carries what a stronger model discovers to a weaker one.

7 Where do the gains come from?



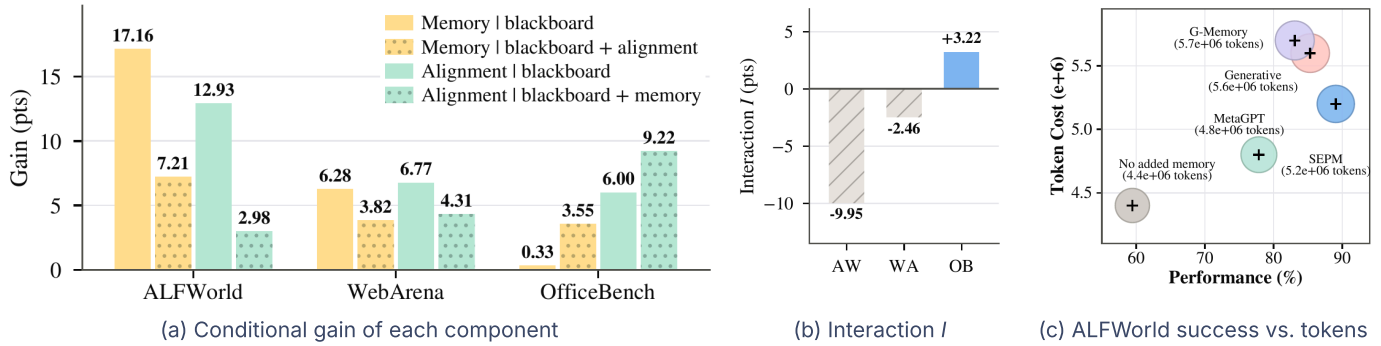
Checkable prerequisites matter

Replacing graph edges with text costs **16.91 / 8.13 / 12.22** points on ALFWorld / WebArena / OfficeBench; **14.92 / 5.91 / 9.34** of them come from executing without the graph.

Evidence-selected edges

Storing every observed precedence costs 2.55–3.73 points and lowers edge precision on ALFWorld from **0.86** to **0.45**.

Figure 7. Ablation under AutoGen. Bars add the blackboard, memory, and alignment in turn; *Full w/o explicit graph* states prerequisites as text in both execution and memory.



(a) Conditional gain of each component (b) Interaction *I* (c) ALFWorld success vs. tokens

Figure 8. Component interaction and cost under AutoGen. $I = s_{BMA} - s_{BM} - s_{BA} + s_B$, where subscripts denote the blackboard (B), memory (M), and alignment (A). Memory and alignment are complementary on OfficeBench (I = +3.22) and partly overlap on ALFWorld and WebArena. SEPM uses fewer tokens than Generative Agents, G-Memory, and LEGOMem on all three main benchmarks.

SELECTIVE RETENTION

80–82% rejected

Most candidate revisions are rejected (133/164, 737/904, 283/354), most often by the dependency gate.

WITHOUT THE DEPENDENCY GATE

≈ 2× revisions admitted

Scores drop by 2.98 / 2.09 / 2.34 points: co-occurring steps must not enter memory.

LEARNING FROM FAILURES

50% → 76%

A premature step exposing a missing prerequisite is recorded in 82% of cases; success on later tasks of that type rises.

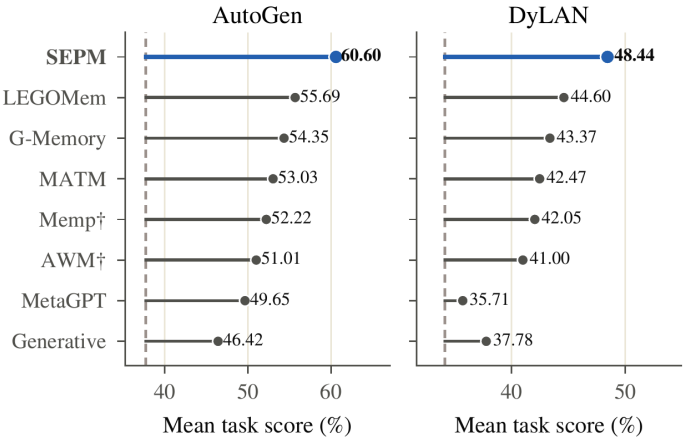


Figure S1. Mean task score over the three main benchmarks for each host. Dashed lines mark the host without added memory.

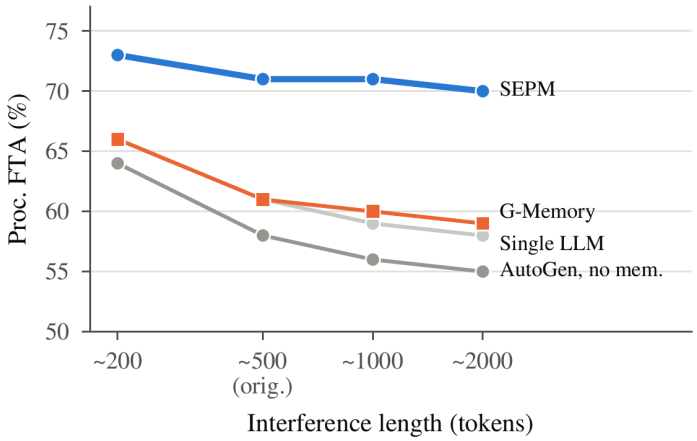


Figure S2. Procedural memory FTA versus interference length (`gpt-5-mini`, one run). About 500 tokens corresponds to the original items.

8 Supplementary results

Full ImplicitMemBench breakdown, sensitivity to the alignment hyperparameters, and run-to-run variation of the ablation. All thresholds were fixed before evaluation and shared across benchmarks.

`gpt-5-mini`	Proc.	Cond.	Prim.	Overall	Δ	Tok./item	Calls/item
Single LLM (no MAS)	61	47	41.27	49.76	+2.15	2.1k	8
AutoGen, no added memory	58	45	39.83	47.61	–	9.6k	31
+ Generative Agents	60	49	42.15	50.38	+2.77	12.4k	38
+ MetaGPT	59	46	40.62	48.54	+0.93	10.8k	34
+ G-Memory	61	52	41.08	51.36	+3.75	13.1k	39
+ AWM†	63	48	39.93	50.31	+2.70	11.2k	35
+ Mem0 (automatic)	60	51	38.77	49.92	+2.31	11.7k	37
+ SEPM (ours)	**71**	**64**	**43.58**	**59.53**	**+11.92**	**12.6k**	**41**
Mem0 + Key Info (oracle)	74	79	33.40	62.13	+14.52	10.1k	32

Table S1. ImplicitMemBench with `gpt-5-mini` (single run). Proc./Cond.: first-attempt accuracy; Prim.: priming influence score; Δ relative to AutoGen without added memory. Second-best non-oracle score underlined.

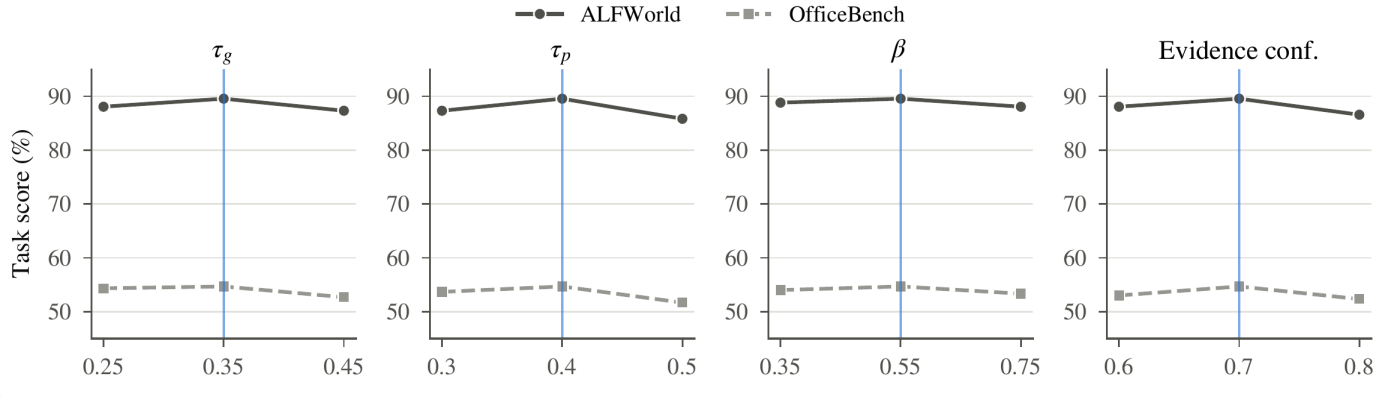


Figure S3. Sensitivity to alignment hyperparameters under AutoGen (one run), varying one parameter at a time. Vertical lines mark the defaults.

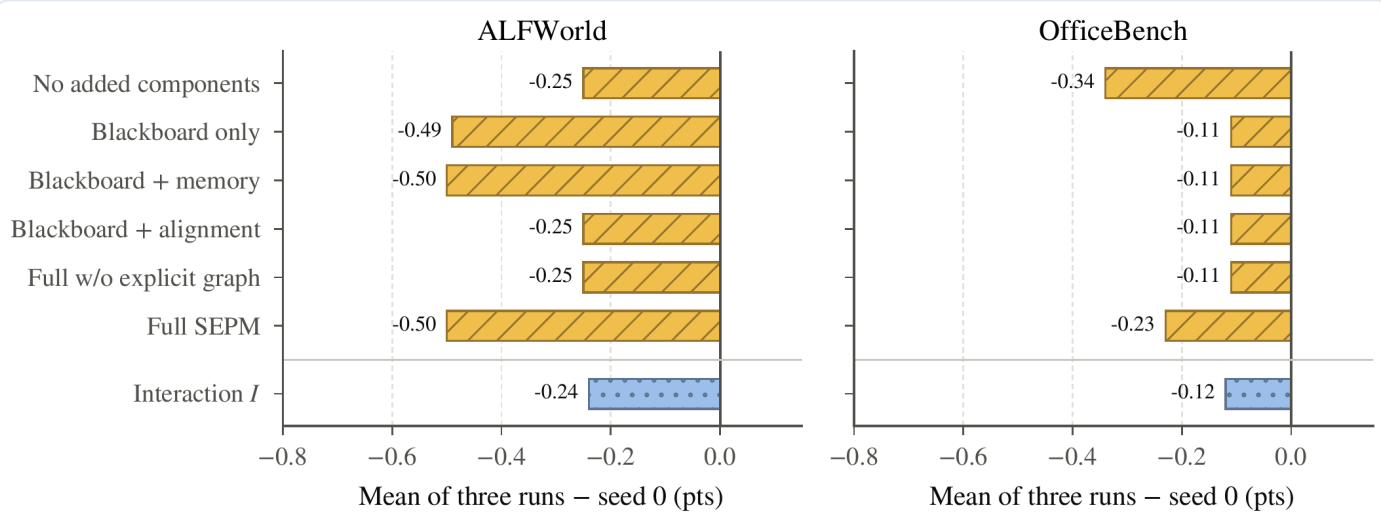


Figure S4. Three-run mean minus the seed-0 run for each ablation configuration and the interaction I (pts). WebArena uses one run and is omitted.

9 Code: installation and usage

The core library depends only on Pydantic and runs without an API key. All commands assume a Linux shell with Python ≥ 3.10. The code is provided in the supplementary material and will be publicly released.

1 Install

```
cd SEPM                                          # unpacked supplementary archive
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"                          # add ",eval" for the benchmark harnesses
cp .env.example .env                             # set OPENAI_API_KEY and OPENAI_BASE_URL
```

2 Run the end-to-end example and the checks

```
python examples/end_to_end.py                    # plan, validate, publish, retrieve
sepm --db sepm_demo.db retrieve "verified catalog results"
pytest -q && ruff check src adapters tests       # unit and integration-contract tests
sepm-benchmark                                   # deterministic mechanism checks, no LLM
```

3 Plug SEPM into an agent host

SEPM and the included baselines (official G-Memory, Generative Agents, and MetaGPT memory classes; Mem0) implement one plug-in contract, so a host swaps memories without other changes.

```
from sepm.llm_client import LLMClient, UsageMeter
from sepm.plugins.sepm_plugin import SEPMPlugin

memory = SEPMPlugin(database_path="pool.db", task_family="my_tasks",
                    client=LLMClient(), meter=UsageMeter())
memory.begin_task("task-1", goal)
prompt_block = memory.context("agent-a", goal)                  # procedures + warnings
corrections  = memory.record("agent-a", action, observation)    # blackboard + alignment
memory.end_task(success=True, reward=1.0)                       # outcome, trials, extraction
```

Repository layout

```
SEPM/
├── src/sepm/          # core library
│   ├── plugins/       # memory plug-in contract
│   ├── hosts/         # AutoGen-style team host
│   ├── bench/         # harnesses + report
│   └── ablation/      # component ablation
├── adapters/          # ALFWorld, OfficeBench
├── scripts/           # experiment suites
├── examples/          # end-to-end example
├── tests/             # unit and contract tests
├── evaluation_matrix.json
└── environment.yml
```

Default hyperparameters

Retrieved procedures K	5
BM25 k_1, b	1.5, 0.75
Goal weight β, threshold δ_g	0.55, 0.35
Plan threshold δ_p	0.40
Evidence confidence	0.70
Safety-risk threshold	0.50
Dependency support c_{dep}	0.65
Qualified trials m	1
Cost weight λ	1.0

One configuration for all benchmarks, fixed before evaluation.

Code released under the MIT License